Supercharging A/B Testing at Uber

- uber.com/en-PL/blog/supercharging-a-b-testing-at-uber

July 21, 2022

Introduction

“Immensely laborious calculations on inferior data may increase the yield from 95 to 100 percent. A gain of 5 percent, of perhaps a small total. A competent overhauling of the process of collection, or of the experimental design, may often increase the yield ten- or twelve-fold, for the same cost in time and labor. To consult the statistician after an experiment is finished is often merely to ask him to conduct a post mortem examination. He can perhaps say what the experiment died of. To utilize this kind of experience he must be induced to use his imagination, and to foresee in advance the difficulties and uncertainties with which, if they are not foreseen, his investigations will be beset.” (R. A. Fisher’s Presidential address to the 1st Indian Statistical Congress)

While the statistical underpinnings of A/B testing are a century old, building a correct and reliable A/B testing platform and culture at a large scale is still a massive challenge. Mirroring Fisher's observation above, carefully constructing the building blocks of an A/B platform and ensuring the data collected is correct is critical to guaranteeing correctness of experiment results, but it's easy to get wrong. Uber went through a similar journey and this blog post describes why and how we rebuilt the A/B testing platform we had at Uber.

Uber had an experimentation platform, called Morpheus, that was built 7+ years ago in the early days to do both feature flagging and A/B testing. Uber outgrew Morpheus significantly since then in terms of scale, users, use cases, etc.

In early 2020, we took a deeper look at this ecosystem. We discovered that a large percentage of the experiments had fatal problems and often needed to be rerun. Obtaining high-quality results required an expert-level understanding of experimentation and statistics, and an inordinate amount of toil (custom analysis, pipelining, etc.). This slowed down decision-making, and re-running poorly conducted experiments was common.

After assessing the customer problems and the internals of Morpheus we concluded that the core abstractions supported only a very narrow set of experiment designs correctly, and even minute deviations from such designs resulted in incomparable cohorts of users in control and treatment and compromised experiment results. To give a very simple example, while gradually rolling out an experiment that was split 30/70 between control and treatment, due to peculiarities of rollout and treatment assignment logic it would be ok to roll it out to 10% of users but not to 5%. Furthermore, the system was not able to support advanced experiment configurations needed to support Uber's diverse use cases, or other advanced functionality at scale such as monitoring/rolling back experiments that were negatively impacting business metrics. So we decided to build a new platform from scratch with correct abstractions.

What Did We Want to Achieve in the New System?

Our goal was to allow the company to run a *wide variety of experiments* in an *agile manner* with *high quality*.

High Degree of Quality Assurance

The role of the experimentation system in the company is to empower decision-making by providing credible insights into the consequences of decisions. A good experimentation system does so by providing consistently correct results, which ensures that:

1. The decisions taken are based on good information and are therefore (hopefully) good decisions
2. The results are universally trusted, allowing teams to align on the ground truth quickly and act on it without endlessly reinvestigating surprising findings, rerunning faulty experiments, and second-guessing decisions

The way Morpheus worked made it hard to ensure correct results in anything other than simple experiments. Combined with Uber's diverse and complex experimentation requirements, this resulted in a lot of questionable data. This led to significant wasted effort: manual investigations and analyses, as well as experiment reruns, were common, slowing down development and diverting attention from other priorities. Sometimes problems in experiment results went unrecognized.

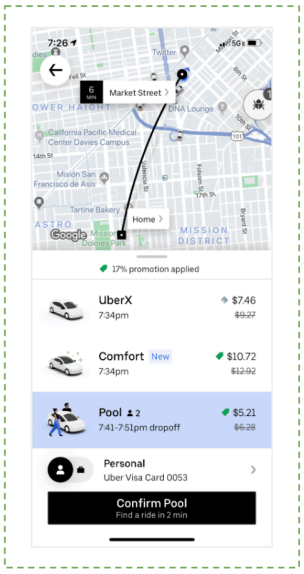
The new system should provide guaranteed correct results: no matter the experiment design chosen by the user, anyone should be able to believe the results of the experiment without the need for custom verification, deep understanding of statistics, or detailed knowledge about the platform.

Beyond statistical correctness, system reliability is a must for an experimentation platform, since it is so central to the way our company operates. Our existing system was developed 7+ years ago as an *optional dependency* in the software stack. However, over the years, feature flagging became the norm and the experimentation system became a *required dependency* for mobile apps and backend services. This transition happened in the mindset of the users, but the system didn't keep up with this evolution. The clients failed closed when the backend didn't respond, leading to all mobile apps and backend services going hard down upon failures in the experimentation stack; we had encountered multiple such outages over the years. We wanted our new system to bake in appropriate safeguards to ensure Uber services are resilient to failures in the experimentation system.

High User Productivity

Uber is primarily experienced by end users via our mobile apps. But the nature of Morpheus made it difficult to experiment on these apps. The programming model used in Morpheus was to specify the treatment groups in the client code (see the diagram below). So changes to experiments—adding a new treatment group or changing an existing one—would require a build-release cycle slowing us by 2-4 weeks. To address this we wanted to decouple experimentation from code changes as much as possible so experiments could be created/deleted/changed without waiting for mobile build/release. Furthermore, we wanted to simplify the client interface and hide the complexity of experiment configuration and serving from the clients.

Client



Experiment setup in mobile

```
# current setup
// Option #1
if (cachedExperiments.isInTreatmentGroup("BUTTON_COLOR_EXPT", "BLACK") {
        color = "FF000000";
else if (cachedExperiments.isInTreatmentGroup("BUTTON_COLOR_EXPT", "BLUE") {
        color = "FF0000FF";
}
// Option #2
color = cachedExperiments.getStringParameter("BUTTON_COLOR_EXPT", BUTTON_COLOR));
```

```
If we want to experiment between BLACK and GREEN colors in future,
- In Option #1, we need to add a check for GREEN color and change BUTTON_COLOR_EXPT to
a new experiment to avoid flicker.
- In Option #2, we need to change BUTTON_COLOR_EXPT to a new experiment to get a
correct read on impact.    MOBILE CODE CHANGE - WAIT FOR RELEASE CYCLE
```

Figure 1: Experiment Aware Pseudocode

A second, related problem was the fragmented configuration stack created during Uber's hypergrowth. We had a configuration system for experimentation and mobile feature flagging, and another stack for backend configuration and backend feature flagging. This had fundamentally fragmented our mobile and backend user workflows, which made it difficult to do experiments that crossed mobile and backend services, required duplicate efforts to add features related to security/compliance/etc., and made it hard to debug issues that arose due to subtle differences in the systems. Moreover, fragmentation meant higher maintenance costs.

Third, experiment analysis limitations were some of the biggest sources of toil for our data scientists. The analysis ecosystem was built to support only user-randomized experiments. Doing analysis on any new randomization units would require custom pipelines and setup, which led to multiple incarnations of the pipelines in different orgs and inconsistencies coming from them. The metrics used in the analysis were not standardized either. Users used their own bespoke metrics that were different and incomparable across orgs. It was difficult for leadership to reason about and compare the impact of experiments across orgs, or even consistently evaluate experiments that crossed org boundaries, which is increasingly the norm as Uber becomes a unified platform with intersecting business lines. Moreover, because the system often produced systematically imbalanced control/treatment cohorts, developing analysis functionality further was not feasible (e.g., functionality such as monitoring experiments for negative impact that was previously attempted was doomed from the start).

Flexibility to Support a Variety of Experiment Designs to Match the Needs of Diverse Product Development at Uber

While some experimentation use cases can be addressed with a simple 50/50 fixed experiment design, many need more sophisticated setup. A key requirement in designing the new system was supporting a wide variety of use cases encountered at Uber, as well as potential future needs.

- Experiments often need to run with complex logic that determines when and where an experiment is rolled out and what proportion of the users can get the new product while it is being experimented on. This logic sometimes needs to change during the experiment due to external factors, and sometimes these changes over time are a key part of the experiment design.
- Experiments need to be run at different levels of granularity; the types of units our users need to randomize on can vary according to use case.
- Experiments need to be run across backend, mobile, web surfaces, or their combinations.
- There is a broad need for hierarchical experimentation, with use cases like:
    - holdouts spanning multiple features with customizations
    - dependent experiments and feature flags
    - traffic splitting across different experiments
    - multiple holdouts for different experiments, etc.
- In some situations, the same feature needs to be experimented on independently in different regions/apps/operating systems/etc.
- All of this flexibility should be satisfied without creating client code complexity.

Beyond the functionality noted above, the new system design should be able to support a wide range of future extensions without requiring major re-architecture.

Architecture

In this section, we'll describe the core concepts in the architecture of the new platform.

Parameter – Decouple Code from Experiments

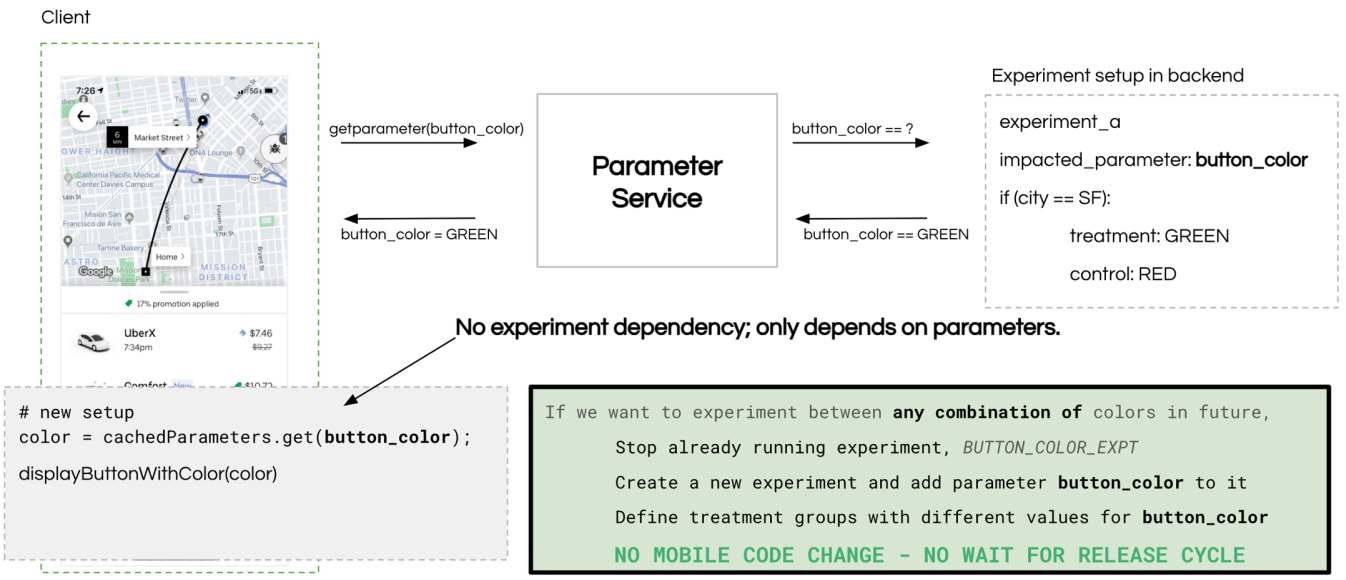


Figure 2: Config Driven A/B Pseudocode

Parameter decouples code from experiments. Instead of referring to the experiment name or treatment group in the client codebase (mobile or backend), the code branches on parameter values. Parameter always has a safe default value (usually equivalent to the "control" path),

which ensures that the clients operate smoothly if a new overridden value doesn't exist or isn't received from the client due to network issues.
Experiments are set up to override values of parameters in the backend. Parameter is the only concept visible to the client—any number of experiments can be set up in the backend to give different values to a given parameter or set of parameters, but the client is unaware of those experiments. Different clients might receive different overridden values of the parameter based on the *context* passed during the call. A backend service delivers parameter values to the client based on the passed context and the client code relevant for the branch of the parameter value is executed. If the experimenter decides that a substantially different experiment design is needed from the one originally launched, they can just disable the current experiment and set up a new one on the same parameters—no code changes required. Similarly, new experiments can be run on the same parameters after the old experiments ended, in case new ideas need to be tested.

The diagram above describes the call flow between the client and the server and the relationship between parameters and experiments.

Unified Configuration and Experimentation Stack

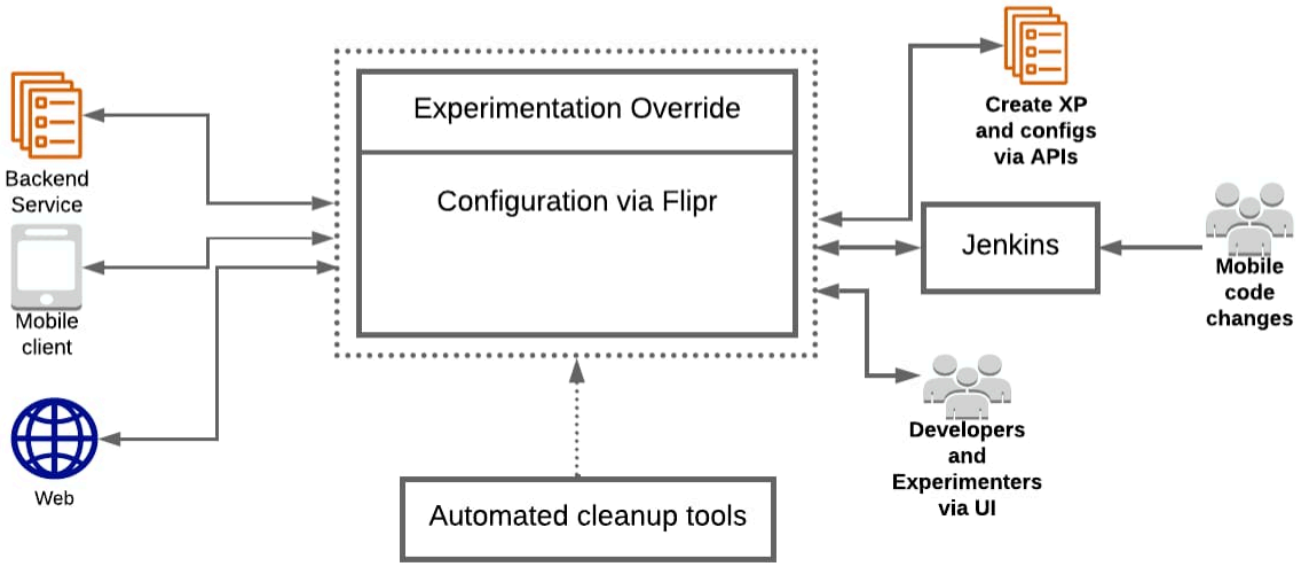


Figure 3: Unified A/B & Remote Config Architecture

We chose to build experimentation as an overriding layer on top of Flipr, which is the system used for backend configuration at Uber. Parameters live in Flipr, and if there's an experiment on top of the parameter, experimentation systems are called automatically and the overridden value of the parameter is delivered from the experimentation system. Otherwise, the default parameter value is served. This decision unified mobile, backend, and experimentation configuration all into one system, enabling reuse, unifying developer workflows, and making it easy to run experiments across different surfaces.

Experiment

Our experimentation system is based on a single concept of an *experiment*. There are no other structures: no separate holdout constructs, traffic splitters, layers/domains, etc. More complex experimentation functionality is built out of *experiments* as explained below.

At its core, an *experiment* consists of 3 key parts:

- *randomization*: the way units are mapped into treatment groups
- *treatment plan*: a mapping from context and unit's randomization (treatment group) into actions (values for parameters)
- *logs*: an auxiliary construct that records additional information in the experiment

Randomization

This is experimentation 101: we need something random and independent of any information about experimentation units, environment, or anything else; this randomness will serve to differentiate the actions we take on otherwise identical experiment units in otherwise identical circumstances.

We randomize units by hashing their identifiers with a salt determined by the experiment key in a given experiment. Experiment keys are unique, which ensures that all experiments are randomized independently of each other and anything else, as long as no other system at Uber is using the same hashing logic.

More specifically, we construct a unit's *bucket* as the residual in integer division of the unit's hash with a certain modulus specified in the experiment (typically 100). By construction, a unit's bucket in a given experiment never changes—and is easily replicable.

Treatment groups are sets of buckets. In practice, we organize treatment groups into a tree, with each treatment group (node in a tree) being a contiguous range of buckets. For example, in an experiment that splits units into 100 buckets, the root node would be all_units with buckets [0..99], and it could be split into control [0..49] and treatment [50..99]. Treatment groups can be further split as necessary, so for example treatment [50..99] could be split into t1 [50..59], t2 [60..69], and t3 [70..99].

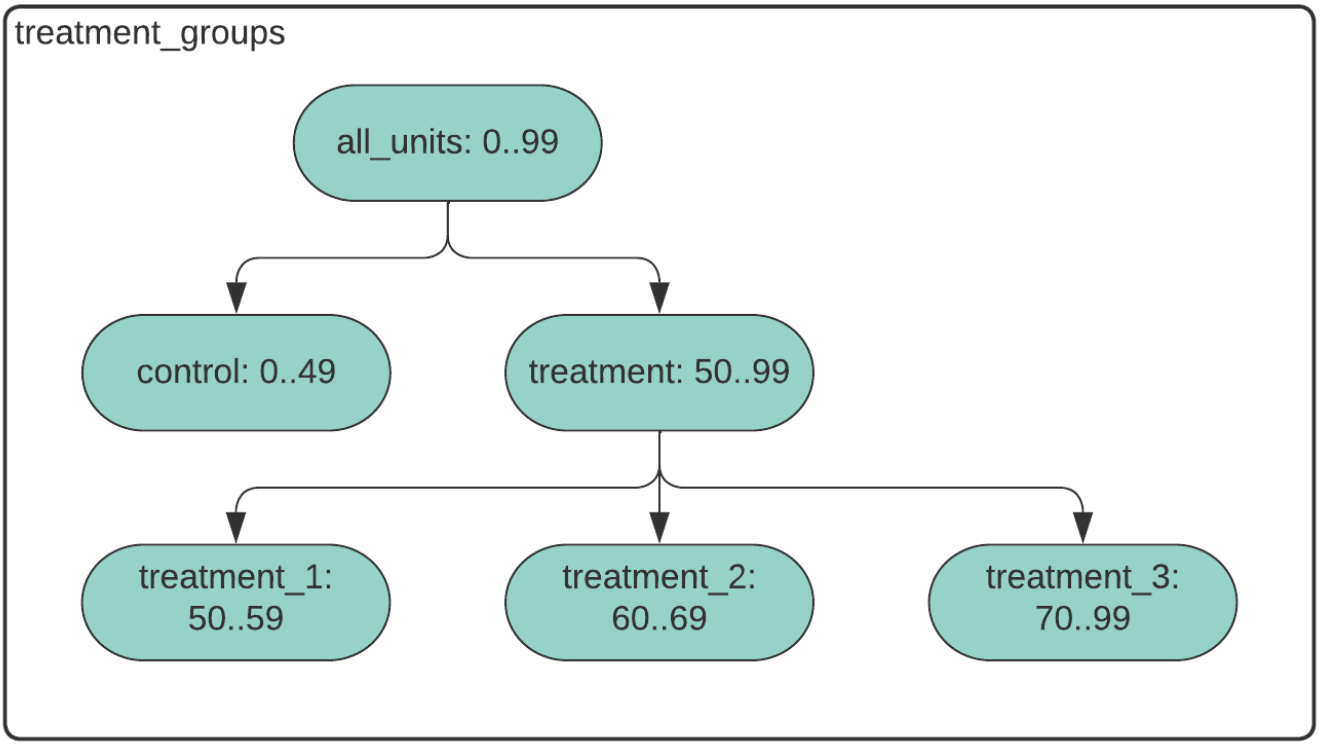


Figure 4: Treatment Group Assignment

The ability to split treatment groups allows for more complex experimental designs that incorporate changes during experiment execution (e.g. giving control experience back to more users while the experiment is running).

Treatment Plan

The *treatment plan* specifies what is done in each *context* to each treatment group.

Context is the knowledge we have about an experimentation unit. It could include geographical information such as the city or country the user is located in, device characteristics such as the operating system (iOS or Android), as well as potentially anything else available at the time of experiment evaluation.

Actions in our system are simply the parameter values we return, for the parameters *impacted* by the experiment (an experiment has one or more *impacted parameters*).

So for example, we can contemplate an experiment run in San Francisco on the color of a particular button. In this case, an experiment would declare that it operates on parameter button_color, with control units that match to context.city = San Francisco getting button_color = green and treatment units in the same context getting button_color = black. All units get the default button color when they are not in San Francisco, which can be green, black, blue, or anything else.

Some aspects of context can be randomly generated as well: for example, we use (pseudo-)randomly generated rollout buckets, with rollout randomization independent of treatment assignment, in order to enable gradual rollout of experiments.

Multiple Experiments on the Same Parameter

As part of the treatment plan, experimenters can declare what parts of the context space the experiment needs to *capture* for its exclusive use, and what parts of context can be used to run other experiments on the same parameters. As such, different independent experiments can be run on the same product feature in different cities, different device operating systems, or other subsets of context as needed, as long as the experiments on the same parameters do not overlap. We leverage a custom logic engine to detect at configuration time whether any two experiments impacting the same parameter overlap, and allow experiment create or update operations as long as they do not.

Logging

Correct data is crucial for powerful experiment analysis. The goal of experiment logs is to identify the first time the unit was exposed to a given experiment—as in, the unit was in a situation where different parameter values would have been served to units in different treatment groups. This is accomplished by emitting logs upon accessing parameters overridden by an experiment. In a distributed system, it's not easy to know which is the first exposure, so we emit logs upon every access of the parameter and dedup the logs later in the data pipelines.

Logs are not emitted if the unit is in a context where the same parameter value is specified for all treatment groups—as in, there's no divergence in experience for the units. This is done because if there's no divergence in experience, there's no impact to measure and hence logs are unnecessary. Finally, log emission is transparent to the client code.

While the idea of logging is simple, logging data without any drops is an engineering challenge, especially at scale (>>1M msgs/sec). We needed careful tuning and organization of the logging code to ensure that we minimize the cases of logging the data without giving experience to the users or vice versa. However, once the data is logged, the actual analysis of the experiment based on the logs is completely decoupled from the serving and configuration layer. As long as the configuration and serving layers log and deliver cohorts where treatment assignment is exogenous, analysis can count on that and focus on extracting the most value from a valid experiment.

Correctness

While a formal exposition of the properties of our system goes beyond the scope of this blog post, it is worth noting that in the architecture outlined here, regardless of what experiment design (configuration) a user chooses, all treatment groups will be comparable to each other—there will not be systematic differences between them except for the actual treatment effects. This is guaranteed as long as anything that describes context and/or determines unit inclusion in analysis (e.g., first log) is independent of treatment assignment up to the time of first exposure to differentiated experience, which in our system is return of differentiated parameter values.

Parameter Constraints

The section above describes what a single *experiment* is. How do we bridge the gap from this simple structure to complex hierarchical designs? *Parameter constraints* is the answer.

Just as an experiment can reference location city or device OS as a constraint in its treatment plan (e.g., if the experiment is only run in the US on iOS), it can also reference other *parameters* as part of that logic. For example, a user can choose to only run the experiment if a constraint specifying parameter_a == true is satisfied. Given that parameters can be controlled by experiments, this opens the door to a wide variety of nuanced setups which address real world use cases—and since experiments are randomized independently of each other, even complex experiment setups remain correct. The only restriction we impose is that we avoid circular dependencies (ensure that the dependency graph is a DAG).

Some of the use cases this enables are:

- *Traffic splitting*: Combining this feature with the ability to run multiple experiments on the same parameters, we can split traffic into random slices with a traffic_splitter experiment controlling traffic_slice parameter, and run distinct downstream experiments that each incorporate a constraint like traffic_slice == “A” and only control parameter values on their slice.
- *Holdouts*: This primitive also enables holdouts. We can have an experiment that creates a holdout at Uber level (by setting a parameter uber_holdout = true), and then holdouts at org level for units not captured by the uber_holdout, and further smaller holdouts at team level. Experiments that are used as holdouts can have complex configurations themselves, such as specifying that Uber employees or executives are not included in the holdout so they can experience product changes quickly, or only applying holdout logic in certain geographic regions.
- *Dependent experiments/feature flags*: Sometimes an experiment can only be run if a certain feature is enabled – but that feature itself might still be in an experimentation phase, or have complex rollout logic for where it is and is not available. In this case, the downstream experiment can reference the parameter that enables the main feature, and only run when that parameter is set to true.

Generalized Data Pipelines

Given that adding new units of randomization was a common use case, we designed our pipelines to be generic without assuming anything about the units of randomization. The experiment logs, as well as the pipelines, produce the same data for all randomization units: experiment key, unit_id, timestamp when the unit was exposed to a particular experiment, the context that was passed when the unit was exposed, name of the parameter that was accessed, and some miscellaneous metadata. All the analysis libraries also perform the analysis on this generic data without assumptions about units in the pipeline layer. Only at the analysis time, the user would choose the metrics that are relevant for that unit to join on the

underlying data. This decision to keep the pipelines generic made it a matter of simple config change to introduce new experiment units and leverage the rest of the infrastructure and pipelines without changes.

Generalized Analysis Engine

Morpheus had a UI to generate and view analysis results, but the stats package behind that was written in Scala and hidden away in the service, making it hard for data scientists to use the same package for custom analysis, which led to duplicate and non-standard statistics for analysis.

In the new system, we built the stats package as a Python package and built the analysis UI layer to invoke this package underneath. We shared the same Python package with the data science community internally so it can be used for exploratory analysis in Jupyter notebooks or for doing deeper analysis that'd be hard to do in the UI layer. While the UI layer still powers most of the analysis use cases, offering the same analysis engine to be accessible via DS native toolchain enabled power users to do more complex analysis, and increased transparency to users in terms of what the system does.

We integrated the analysis package with uMetric as the only source of metrics. Given that experimentation is the primary use case of metric usage, this enabled us to solidify uMetric as the single source of metrics in the company and further increase metric sharing/standardization across users and teams.

In order to support the wide variety of experiment analysis needs across our teams, we offer highly flexible on-demand analysis that allows users to answer granular questions about the effect of their innovations.

There are two key parts to every analysis: the *cohort* being analyzed and the *metrics* that are analyzed for that cohort.

The cohort is the set of units that are the focus of a given analysis. We broadly support two approaches to cohort definition:

- *Logged cohort*: based on the experiment logs (e.g., "units that entered the experiment between March 1st and March 30th")
- *Logless cohort*: cohorts defined in a way that does not depend on logs (e.g., "users that took at least 1 trip in 28 days before the experiment started")

Logged cohort analysis is perhaps the more standard approach, both for experiments run at Uber and across the industry. It allows users to precisely focus on units affected by the experiment, and generally gives precise results.

Logless cohorts, on the other hand, allow us to have a fallback way to analyze experiments in case some logs were lost due to outages and other issues, as well as provide a way to analyze experiment results on cohorts that are directly comparable across experiments. Such analyses can be less powerful, but are extremely robust given our architecture.

In order to support fine-grained insights, we provide a lot of flexibility for users to segment both cohorts and metrics.

Cohorts can be segmented on essentially any data that is defined up to the time of the unit's first entry into the experiment; this allows us to provide results such as impact on iOS vs Android users, and new vs existing users, while guaranteeing that even with segmentation the cohorts remain comparable between different treatment groups.

For a given cohort or segment of a cohort, metrics can also be segmented by leveraging metric dimensions, so we can estimate impact on completed trips on UberX vs UberXL. Because of metric standardization via uMetric, the metric impact results reported in experiments are consistent with the way metrics are computed in reporting dashboards relied on by teams across the world.

Reliability: SDKs, Fallback Parameter Value

Given that the experimentation system had firmly established itself as a requirement dependency in mobile and backend development, we wanted to make a step-function change in the reliability of this system to eliminate a class of high impact outages we had seen before. We did a number of things to achieve this.

First, as mentioned before, parameters always have a *safe default value*. For mobile apps, this default value is shipped with the mobile app itself. And for backend services, Flipr serves a default value locally (on the host). So in the event of network failures or delays, the clients have a value that leads to a safe code path.

Second, we built SDKs for all languages (Go/Java/Android/iOS/JS/etc.) and clients (web/mobile/backend) used at Uber. Mobile SDKs have a cache for the previous payload received from the backend; the mobile apps fall back to this cache, and if the cache doesn't exist and the backend is unavailable, they fall back to the default value. In the case of the backend, the clients fall back to Flipr default value (served locally) in case experimentation could not be reached or if there is a timeout. These multiple layers of fallback help improve reliability significantly.

Third, the SDKs automatically emit experiment logs just before a fork in the experience. This automated logging eliminated toil as well as bugs that come from manual logging. We have implemented a host of optimizations over time into the SDKs (caching/deduping/batch logging/etc.), which benefit all users of experimentation with no extra effort.

Fourth, the SDKs support a feature called prefetching parameters—this enables fetching a batch of parameters upfront to save latency, but doesn't emit logs until they are *accessed* at a later point in time (decouples fetching from logging). This addresses the use case where the feature being experimented on is latency sensitive at the point where parameter needs to be accessed, but less latency sensitive earlier on in the user flow. Incurring latency costs before a feature needs to be shown while providing instantaneous access to parameters when they are actually needed—and automatically emitting logs at that point—allows for responsive user experience and correct and powerful experimentation at the same time.

Challenges and Learnings

One team: Building an experimentation platform requires a diverse set of skills. The system as a whole needs to be correct from a statistical perspective, it needs to be scalable and reliable, and the user experience needs to be smooth to ensure user productivity. System design decisions cannot be easily separated into eng and DS: minute details of error handling and performance optimizations in serving can have substantial implications for statistical properties of results, necessitating DS involvement —on the other hand, details of the implementation of statistical analysis matter for reliability, how well it scales to larger amounts of data, how much compute resources are required to carry it out at scale, etc. We succeeded by working as truly one team, in particular with tight integration and collaboration between engineering and data science. We have common architecture reviews, joint code ownership, one team roadmap/priorities, and common social events.

Partner integrations: The experimentation platform has close integration with a host of partner systems: push/email communication, targeting, segmentation, campaign management, no-code workflows, and many others. These systems contribute close to 40% of all experiments. Building out integration with all these systems early on made it easier for customers who often tend to use multiple of these systems in conjunction to move over easily to the new system. Planning the collaboration and prioritizing this integration paid off by significantly reducing the adoption time.

Communication: For such massive and foundational changes to a platform that's used by most of Uber on a day-to-day basis, communicating and bringing the customers along is crucial. To do this we did a number of things:

- Listening sessions during the problem discovery and design phase to understand pain points that needed addressing
- Presented new architecture and functionality broadly early on to key customers, explained the customer benefits, and iterated on the design based on their feedback
- Conducted demo sessions when launching the platform to facilitate understanding and smooth adoption of the new platform
- Regular listening sessions to get feedback on the product – altered our roadmap and reprioritized things to address the pressing needs

Adoption, Migration, and Deprecation: Thinking through the strategy for new platform adoption, migration of relevant XPs to the new platform and deprecation of tens of thousands of stale XPs (that were years old) in Morpheus from early on was very critical. We phased out the deprecation into multiple phases, starting with the highest risk experiments and expanding into other experiments. We had custom programs focused on deprecating specific categories of experiments. We built a suite of tools to migrate, monitor, and disable a large portion of stale experiments. This reduced the work decentralized teams have to do significantly and limited them to only focus on the experiments that could not be auto migrated or centrally deprecated. That was still very painful for decentralized teams. Adoption of the new platform ran in parallel.

Closing

After over a year's work, we built a strong foundation for experimentation and feature flagging ecosystem at Uber. We have transitioned all of Uber—over 2000+ developers, 15+ integrated partner systems, 10+ mobile apps, 350+ services—to the new system. We deprecated over 50,000 stale experiments in Morpheus.

In our next phase, we are working on building many functionalities to take the usability, user experience, performance, automated monitoring, automated rollout, and our overall ability to experiment and ship products more quickly to the next level. We plan to share more details in future blog posts. The experimentation team is actively hiring. If you are interested in helping us build a state of the art experimentation platform for Uber's scale, please explore opportunities here.

Acknowledgments

This work was a result of collaboration with a number of teams. We'd like to thank the following folks for their contributions: Akshay Jetli, Ali Sakr, Alicia Savelly, Ameya Ketkar, Andrew Wansink, Andy Maule, Azarias Reda, Chenbo Xie, Chenyu Qiu, Christian Rauh, Dan Ho, David Malison, Elenora Avanzolini, Eric Chao, Erica Shen, Harshita Chandra, Jennifer Chou, Jun Yan, Junya Li, Kevin Chin, Laith Alasad, Lauren Motl, Lazaro Clapp, Likas Umali, Lisa Kuznietsova, Mahesh Hada, Manish Rao Sangi, Matt Bao, Mikhail Lapshin, Murali Krishna Ramanathan, Mustafa Turan, Nada Attia, Neha Bhangale, Rafael Dietrich, Rahul Chhabra, Rahul Deshpande, Raj Vardhan, Rob Mariet, Sarin Swift, Scott Graham, Senjun Fan, Shirley Zhu, Sidharth Eleswarapu, Tseten Dolkar, Vlady Rafael, Wei Xu, Will Friebel, Yingchen Liu, Yogesh Garg, You Lyu, and Yun Wu.

The headline image is covered by a CC BY-SA 3.0 license and is credited to Amitchell125 at English Wikipedia

Sergey is a Senior Manager leading the Experimentation Science team. As part of this role he has led the re-architecture and rebuild of Uber's experimentation platform, and prior to experimentation Sergey worked on subscriptions products at Uber.

Krishna Puttaswamy is a Distinguished Engineer at Uber. He is involved in solving critical data and experimentation problems at Uber. Over the last two years, he led the re-architecture and rewrite of Uber's experimentation platform. He currently leads re-architecture of Driver domain data models and services.

Luke Duncan is a Senior Manager at Uber leading the Product Intelligence teams including Experimentation, uMetric, and Marketplace tools. In the last two years he led the rebuilding of Ubers experimentation platform and data ecosystem.

Deepak is a Principal Engineer at Uber. He currently works on Uber's B2B products and platforms with a focus on re-architecting several core entities and systems to unlock both engineering and business efficiency. Previously, as a senior IC of the experimentation platform at Uber, he worked on several aspects of the platform that are used by every team in Uber to conduct effective and accurate A/B experiments. Deepak has expertise building scalable,

reliable software systems across a wide array of domains such as Cloud-Agnostic Machine Learning infrastructure, Cloud File storage, Configuration Systems, Virtualization, and Disaster Recovery. He earned his PhD in Computer Science from Purdue University, where his doctoral thesis title was ‘Highly Available Storage Systems’.
Arun Babu A S P is a Senior Staff Engineer at Uber, specializing in Mobile. Since early 2017, he has been instrumental in designing core frameworks across Maps, Networking, App Health, and Experimentation domains at Uber. He currently leads a cross-geographical team from the US. He has been an architect for large-scale mobile apps and SDKs for over a decade.
Posted by Sergey Gitlin, Krishna Puttaswamy, Luke Duncan, Deepak Bobbarjung, Arun Babu A S P

Category:
Data / ML

Engineering